Documentation Duality and AI Collaboration

Documentation Duality and AI Collaboration I

Every directory at every level of the repository hierarchy contains two documentation files:

Documentation Duality and AI Collaboration I

- `README.md`: Human-readable overview, quick-start instructions, and directory structure.
- `AGENTS.md`: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—`AGENTS.md` files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding `AGENTS.md` files, keeping documentation synchronized with implementation. Lau and Guo's survey of 90 AI coding assistant systems (Lau and Guo 2025) identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

The template additionally includes `CLAUDE.md` at the repository root, providing system-level instructions for AI coding assistants—architectural principles, testing requirements, and naming conventions that apply globally. This creates a three-tier documentation architecture: per-directory `AGENTS.md` for local context, root `README.md` and `CLAUDE.md` for global constraints, and `README.md` for human navigation.

Agentic Skill Architecture

Agentic Skill Architecture I

The Documentation Duality standard addresses human and AI navigation at the directory level. A complementary layer operates at the *module* level: every infrastructure subpackage carries two additional machine-readable files that transform it from a passive code library into an active, protocol-aligned skill endpoint.

The Three-Tier Skill Protocol I

`template/` implements a progression of agent-facing documentation, escalating in specificity from global constraints to module-level API contracts:

The Three-Tier Skill Protocol I

Tier	File	Scope	Purpose
1 — System	`README.md`	Repository root	Global architectural principles, Zero-Mock policy, naming conventions
2 — Structure	`AGENTS.md`	Every directory	Local file inventories, API surfaces, integration patterns, architectural constraints
3 — Skill	`SKILL.md`	Every infrastructure module	Machine-parseable skill descriptor: module name, description, key imports, usage pattern

Tier 1 and Tier 2 have direct analogues in the prompt-engineering literature: system prompts and retrieval-augmented context (Lau and Guo 2025). Tier 3 is novel. The `SKILL.md` files follow a YAML frontmatter + Markdown instruction format precisely aligned with the tool-descriptor schemas of the Model Context Protocol (Anthropic 2024). The following is the actual frontmatter from `infrastructure/rendering/SKILL.md`:

```
---
name: infrastructure-rendering
description: Skill for the rendering infrastructure module
---
```

The Three-Tier Skill Protocol I

An MCP client reading this block immediately knows the module name, its natural-language activation condition ("use for"), and which Python symbols to import. No source-code inspection is required. This is the practical implementation of Toolformer-style self-documented tools (Schick et al. 2023)—rather than a language model learning tool APIs from training data, the APIs are encoded directly in version-controlled, co-located skill files that evolve with the codebase.

Module Skill Coverage I

Each infrastructure subdirectory surfaced by `discover_infrastructure_modules()` carries paired `README.md` + `AGENTS.md`; agent-facing `SKILL.md` manifests exist wherever teams enable Cursor / PAI manifests (regenerated via `python -m infrastructure.skills`). Root-level `PAI.md` summarizes cross-package obligations.

Promotion policy: new Layer-1 directories must ship human + machine-readable docs (`README.md`, `AGENTS.md`) immediately; Tier-3 SKILL assets follow once APIs stabilize.

MCP Server Mapping I

The mapping from `SKILL.md` descriptors to MCP server endpoints is intentional but not yet automated; it represents the principal next integration step. In the MCP architecture (Anthropic 2024), a server exposes three primitive types: **Tools** (executable functions), **Resources** (data the model can read), and **Prompts** (structured query templates). Each `infrastructure` module maps naturally onto this taxonomy:

MCP Server Mapping I

- `infrastructure.llm` → MCP **Tool** (`query`, `apply_template`) + MCP **Prompt** (research prompt templates)
- `infrastructure.rendering` → MCP **Tool** (`render_pdf`, `render_html`) + MCP **Resource** (rendered PDFs as URI-addressable resources)
- `infrastructure.validation` → MCP **Tool** (`validate_pdf_renderin g`, `validate_markdown`)
- `infrastructure.publishing` → MCP **Tool** (`publish_to_zenodo`, `gen erate_citation_bibtex`) + MCP **Resource** (DOI registry)
- `infrastructure.steganography` → MCP **Tool** (`SteganographyProces sor.process`) + MCP **Resource** (hash manifests)
- `infrastructure.search` · `infrastructure.reference` → MCP **Tool** wrappers over literature retrieval + BibTeX handling + MCP **Resource** exports for corpus JSON / `.bib`

MCP Server Mapping I

An agent orchestrating a full research pipeline could, in principle, compose these MCP tools to reproduce the declarative DAG programmatically—discovering capabilities via `SKILL.md` frontmatter, executing them via MCP protocol calls, and consuming their outputs as Resources. The `SKILL.md` files parallel Voyager's skill library (Wang et al. 2023)—Voyager's agent accumulates a growing library of executable Minecraft skills represented as JavaScript functions; `template/`'s agent accumulates a curated library of research pipeline skills represented as YAML-frontmattered `SKILL.md` files. In both cases, the skill representation is machine-readable, version-controlled, and self-describing. Wang et al.'s LLM agent survey (Wang et al. 2024) identifies tool use, planning, and memory as the three fundamental capabilities of autonomous agents; Yao et al.'s ReAct framework (Yao et al. 2023) demonstrates that

MCP Server Mapping I

interleaving reasoning traces with tool actions dramatically improves agent reliability in interactive settings. The `template/` skill architecture maps cleanly onto these three capabilities: the `SKILL.md` descriptors supply the tool-use layer, the declarative DAG of 17 `pipeline.yaml` stages (a default full run executes 10) supplies the planning scaffold, and the per-criterion checkpoint system supplies the memory layer.